Week 4 easier

Week 4: Filter Design I. FIR vs. IIR filters. Windowing methods for FIR design, bilinear transformation for IIR design.

Week 4 easier

Can you map out two 75 min lectures for week 4 (Lectures 7 and 8). The class is a senior/graduate level DSP and ML course. The textbooks are Oppenheim/Schafer "Discrete-Time Signal Processing" and Raschka "Machine Learning with PyTorch and Scikit-Learn"

Gemini Pro:

Designing these lectures for a senior/graduate-level audience requires bridging the rigorous mathematical foundations of Oppenheim and Schafer with the modern, data-driven applications found in Raschka's PyTorch/Scikit-Learn text.

Here is a 75-minute breakdown for both lectures, structured to teach classical DSP while explicitly connecting it to machine learning pipelines.

Lecture 7: FIR Filter Design, Windowing, & 1D Convolutions

Readings: Oppenheim/Schafer Ch. 7; Raschka (Convolutional Neural Networks chapter).
Focus: Mathematical formulation of FIR filters and treating them as fixed-weight convolutional layers.

- **1. FIR vs. IIR: The ML Perspective (10 min)**
- Review linear phase vs. non-linear phase.
- *ML Connection:* Emphasize why linear phase (FIR) is critical when extracting temporal features for neural networks: it preserves the wave shape and prevents frequency-dependent time shifts that could confuse a model.
- **2. The Ideal Filter & Gibbs Phenomenon (20 min)**
- Derive the ideal lowpass filter impulse response: $h[n] = \frac{\sin(\omega_c n)}{\pi n}$.
- Discuss the impossibility of infinite, non-causal filters in real-time systems.
- Demonstrate how abrupt rectangular truncation causes the Gibbs phenomenon (oscillations near the cutoff).
- **3. The Windowing Method (25 min)**

- Introduce windowing as a time-domain multiplication to smooth the truncation: $h_w[n] = h_{ideal}[n] \cdot w[n]$.
- Compare Rectangular, Bartlett, Hanning, Hamming, and Blackman windows.
- Analyze the trade-off in the frequency domain: Main-lobe width (transition band) vs. Side-lobe attenuation (stopband rejection).
- **4. Bridging DSP and PyTorch (20 min)**
- Show how the discrete convolution sum $y[n] = \sum_{k=0}^{M} h\,[k]x[n-k]$ is identical to a 1D convolutional layer (`torch.nn.Conv1d`) in PyTorch.
- *Insight:* Traditional DSP *designs* the weights ($h[k]$) using math (windowing). Machine Learning *learns* the weights using backpropagation.

Lecture 8: IIR Design, Bilinear Transformation, & Data Preprocessing

Readings: Oppenheim/Schafer Ch. 7. **Focus:** Leveraging continuous-time analog prototypes and understanding phase implications in ML preprocessing.

- **1. Analog Prototypes (15 min)**
- Briefly review classical continuous-time (s-domain) filters: Butterworth (maximally flat), Chebyshev I/II (equiripple), and Elliptic.
- Display the magnitude-squared function for a continuous Butterworth filter:

$$|H_c(j\Omega)|^2 = \frac{1}{1 + \left(\frac{\Omega}{\Omega_c}\right)^{2N}}$$

- **2. Domain Mapping & The Bilinear Transform (25 min)**
- Explain the requirement to map the analog s-plane to the digital z-plane while mapping the entire left-half plane strictly inside the unit circle (guaranteeing stability).
- Introduce the algebraic substitution:

$$s = \frac{2}{T}\frac{1 - z^{-1}}{1 + z^{-1}}$$

- **3. Frequency Warping & Pre-warping (20 min)**
- Derive the nonlinear relationship between continuous frequency Ω and discrete frequency ω:

$$\Omega = \frac{2}{T}\tan\left(\frac{\omega}{2}\right)$$

- Walk through the design steps: start with a digital spec, pre-warp to an analog target, design the analog filter, and apply the Bilinear Transform.
- **4. IIR Filters in Scikit-Learn / ML Pipelines (15 min)**
- Discuss when to use IIR filters (e.g., aggressive noise floor reduction prior to feeding a Random Forest or SVM).
- Explain zero-phase filtering (`scipy.signal.filtfilt`), which runs an IIR filter forward and backward to cancel out phase distortion—a technique highly relevant for offline ML dataset preparation.

DSP vs. ML Context Table

You can use this breakdown in your slides to ground the classical DSP concepts in your students' ML vocabulary:

DSP Concept	Mathematical Role	Machine Learning Equivalent/Application
FIR Filter	Feedforward delay line	`Conv1D` layer with frozen or learnable weights
Filter Taps ($h[n]$)	Impulse response sequence	Convolutional Kernel / Weights
Windowing	Smoothing truncation ripples	Kernel regularization / Receptive field bounding
IIR Filter	Feedback (recursive) calculation	Preprocessing step (e.g., smoothing time-series data)
Linear Phase	Constant group delay	Crucial for keeping temporal features aligned for RNNs/LSTMs

---

Week 4

Prompt: Can you create slides and python code for these two 75 minute classes of a DSP and ML course for a combined senior and first year Electrical and Computer Engineering class? The DSP textbook is Discrete-Time Signal Processing by Oppenheim and Schafer. The ML textbook is Machine Learning with PyTorch and Scikit-Learn by Raschka. Add some homework problems with solutions and a weekly quiz with solutions as well. Please render the math equations in LaTeX.

ChatGPT: (worked for 13 min 19 sec)

Created. I built this as a two-lecture instructor package with the DSP mathematics kept rigorous and the ML connections made explicit rather than treated as analogies only.

Main files

- Complete Lectures 7–8 instructor package
- Lecture 7 — FIR Design, Windowing & Conv1d — Microsoft PowerPoint
- Lecture 8 — IIR Design, Bilinear Transform & ML Preprocessing — Microsoft PowerPoint

Lecture 7 contains **30 slides** and Lecture 8 contains **37 slides**.

Python material

- Lecture 7 Python code
- Lecture 8 Python code
- Lecture 7 Jupyter notebook
- Lecture 7 executed notebook
- Lecture 8 Jupyter notebook
- Lecture 8 executed notebook

Both executed notebooks completed with **no execution errors**.

Homework and weekly quiz

- Homework + weekly quiz with full solutions
- Teaching-flow README

Lecture 7 highlights

The deck develops the ideal lowpass filter from the inverse DTFT rather than just quoting the sinc:

$$h_d[n] = \frac{1}{2\pi}\int_{-\omega_c}^{\omega_c} e^{j\omega n}\,, d\omega$$

leading to

$$h_d[n] = \frac{\sin(\omega_c n)}{\pi n}, \qquad n \neq 0,$$

with

$$h_d[0] = \frac{\omega_c}{\pi}.$$

It then connects symmetry to linear phase,

$$H(e^{j\omega}) = e^{-j\omega(N-1)/2}A(\omega),$$

and therefore to constant group delay,

$$\tau_g = \frac{N-1}{2}.$$

The window section compares Rectangular, Bartlett, Hann, Hamming, and Blackman windows with numerical main-lobe/sidelobe intuition and actual frequency-response plots.

I also added an important nuance to the DSP/ML connection: linear phase is **useful when preserving temporal alignment matters**, but it is not universally required for every ML task.

The PyTorch section explicitly shows why textbook convolution,

$$y[n] = \sum_k h\,[k]x[n-k],$$

requires the fixed FIR taps to be reversed when implemented using the `Conv1d` cross-correlation convention.

The numerical demonstration verifies NumPy DSP convolution against PyTorch with a maximum float32 difference of about

$$3.6 \times 10^{-7}.$$

Lecture 8 highlights

The IIR lecture compares Butterworth, Chebyshev I, Chebyshev II, and elliptic analog prototypes and develops the Butterworth magnitude relationship

$$|H_c(j\Omega)|^2 = \frac{1}{1+\left(\frac{\Omega}{\Omega_c}\right)^{2N}}.$$

The bilinear-transform section develops

$$s = \frac{2}{T}\frac{1-z^{-1}}{1+z^{-1}}$$

and explicitly shows why

$$\mathrm{Re}s < 0$$

maps to

$$|z| < 1.$$

The frequency-warping derivation leads to

$$\boxed{\Omega = \frac{2}{T}\tan\left(\frac{\omega}{2}\right)}$$

and then to the prewarping procedure.

A worked example uses

$$\omega_c = 0.4\pi, \qquad T = 1,$$

giving

$$\Omega_c = 2\tan(0.2\pi) \approx 1.4531 \text{ rad/s}.$$

The resulting digital Butterworth design produces a measured (-3\text{ dB}) frequency of approximately

$$0.3999\pi,$$

which provides a nice numerical confirmation of the prewarping calculation.

The `filtfilt` section also goes beyond simply saying “zero phase.” It shows that forward-backward filtering gives

$$H_{\text{fb}}(e^{j\omega}) = H(e^{j\omega})H(e^{-j\omega}) = |H(e^{j\omega})|^2,$$

so students see that the magnitude response is **squared**, the effective attenuation changes, and the operation is noncausal.

I also included the ML-specific issue of **train/deployment mismatch**: a classifier trained using future-aware zero-phase preprocessing may behave differently when deployed with a causal real-time filter.

All displayed mathematical equations in the decks are rendered from LaTeX, and I used the Microsoft-compatible PowerPoint versions as the primary files.

Week 4, Lecture 7, LaTeX math

Explain in more detail what's going on here and what good answers and/or examples are. Please write all math expressions using LaTeX syntax.

Chat GPT: (Medium)

Please convert all math expressions to LaTeX. Don't change anything else.

Gemini Flash:

Absolutely. Lecture 7 is doing more than teaching the window method. Its deeper purpose is to connect four ideas:

ideal frequency response → impulse response → finite FIR implementation → Conv1d kernel

The lecture explicitly frames FIR design and neural 1-D convolution as the same local weighted-sum operation with different sources for the coefficients: classical DSP designs them; ML may learn them. Below is how I would explain the lecture in class, including what strong student answers should sound like and which examples are especially useful.

Slides 3–6: FIR versus IIR from a DSP/ML perspective

The deck begins with the structural distinction:

FIR ⇒ feedforward only

IIR ⇒ feedback / recursion

An FIR filter of length N has

$$y[n] \sum_{k=0}^{N-1} h\,[k]x[n-k].$$

An IIR filter typically has a recursion such as

$$y[n] \sum_{k=0}^{M} b_k\, x[n-k] \sum_{r=1}^{P} a_r\, y[n-r].$$

The deck emphasizes that FIR filters can have exact linear phase when the taps possess the required symmetry, whereas IIR filters usually achieve sharper magnitude responses with fewer coefficients but generally have nonlinear phase.

Why should ML students care about linear phase?

Suppose an audio transient contains several frequency components. If the filter has linear phase, all passband frequency components experience the same delay. If the filter has nonlinear phase, different frequencies arrive at different times. The magnitude spectrum can look nearly unchanged, yet the waveform can be reshaped. That matters if the downstream ML feature depends on timing, such as:

onset time,

peak timing,

transient shape,

or frame-localized features. The deck makes this exact point and wisely cautions not to oversell linear phase: not every ML task requires it.

Slide 4: Why symmetric FIR taps imply linear phase

Suppose the FIR impulse response has length N and satisfies

$$h[n]h[N-1-n].$$

Its DTFT is

$$H(e^{j\omega})\sum_{n=0}^{N-1} h\,[n]e^{-j\omega}\ .$$

For a symmetric sequence, factor out the midpoint delay:

$$\boxed{H(e^{j\omega})e^{-j\ \ (N-1)/2}A(\omega)}$$

where $A(\omega)$ is real for a real symmetric FIR. That means the phase is approximately

$$\angle H(e^{j\omega}) - \omega\frac{N-1}{2}$$

apart from possible π-jumps when $A(\omega)$ changes sign. Therefore the group delay is

$$\boxed{\tau_g(\omega)\frac{d}{d\omega}\angle H(e^{j\omega})\frac{N-1}{2}}$$

which is constant. The lecture describes this as every passband component being delayed by the same number of samples.

Slide 5: Group delay

For a symmetric FIR with

$$N = 41$$

taps,

$$\tau_g\frac{N-1}{2}\frac{40}{2}\boxed{20\text{ samples}}.$$

The strongest answer to

“What does linear phase mean?” is not

“The filter has no phase distortion.” That is too vague.

A better answer is:

“The phase is linear in frequency, so the group delay is constant.” The best answer is the one already highlighted in the deck:

“Linear phase does not mean zero delay. It means a delay that is approximately constant with frequency.”

Slide 6: What does nonlinear phase do to a transient?

The deck asks a very good concept question: two filters have nearly the same magnitude response, but one has constant group delay and the other does not. What can differ?

A weak answer:

“The phase differs.” True, but not enough.

A good answer:

“The time-domain waveform shape can differ even if the magnitude spectra are almost the same.”

A strong answer:

“Frequency components can emerge with different delays, which can smear or reshape a transient. That can shift peaks or alter onset structure and therefore affect time-localized ML features.” That is exactly the type of explanation I would look for.

Slides 7–9: Ideal lowpass filter

The ideal lowpass response is

$$H_d(e^{j\omega})\{1, \quad |\omega| \le \omega_c,\ 0, \quad \omega_c < |\omega| \le \pi.$$

The deck correctly treats this as the engineering dream: flat passband, zero stopband, infinitely sharp transition. But the corresponding impulse response exposes the price. The inverse DTFT is

$$h_d[n]\frac{1}{2\pi}\int_{-\pi}^{\pi} H_d\,(e^{j\omega})e^{j\omega n}, d\omega.$$

Since the response is nonzero only from $-\omega_c$ to ω_c,

$$h_d[n]\frac{1}{2\pi}\int_{-\omega_c}^{\omega_c} e^{j\omega n}\,, d\omega.$$

For

$$n \neq 0,$$

$$h_d[n] \frac{1}{2\pi} \left[\frac{e^{j\omega n}}{jn} \right]_{-\omega_c}^{\omega_c}.$$

Thus,

$$h_d[n] \frac{1}{2\pi jn} \left(e^{j\omega_c n} e^{-j\omega_c n} \right).$$

Using

$$e^{j\theta} - e^{-j} \; 2j\sin\theta,$$

we get

$$\boxed{h_d[n] \frac{\sin(\omega_c n)}{\pi n}}$$

for $n \neq 0$. At $n = 0$,

$$h_d[0] \frac{1}{2\pi} \int_{-\omega_c}^{\omega_c} 1 \, , d\omega \, \boxed{\frac{\omega_c}{\pi}}.$$

The deck expects senior/graduate students to be able to derive this rather than simply quote it. Why is the ideal lowpass impossible in real time? The resulting impulse response:

$$h_d[n] \frac{\sin(\omega_c n)}{\pi n}$$

exists for infinitely many positive and negative n. So it has two problems. First:

$$\boxed{\text{infinite duration}}$$

A finite processor cannot store infinitely many coefficients. Second:

$$\boxed{\text{noncausal}}$$

Negative-time coefficients would require future input samples in a direct real-time implementation. The lecture explicitly lists those two problems and then motivates the practical solution: truncate, shift, and window.

Slide 10: Center first, then shift

Suppose we want an FIR of order M, so the number of taps is

$$N = M + 1.$$

To preserve symmetry, center the ideal response at

$$\frac{M}{2}.$$

A common finite approximation is

$$\boxed{h[n]h_d\left[n-\frac{M}{2}\right]w[n]}$$

for

$$0 \le n \le M.$$

For

$$N = 51,$$

we have

$$M = 50$$

and therefore the shift is

$$\frac{M}{2}$$

So the linear-phase delay is

$$\boxed{25 \text{ samples}}.$$

The deck emphasizes that this shift changes phase but not the magnitude response. That is an important distinction.

Slides 11–12: Gibbs phenomenon

This is one of the most commonly misunderstood parts of FIR design. Suppose we simply truncate the ideal sinc with a rectangular window:

$$h_w[n]h_d[n]w_R[n].$$

The rectangular window is abrupt in time. Because

$$\text{multiplication in time} \leftrightarrow \text{convolution in frequency},$$

the ideal frequency response gets convolved with the window's frequency response. The rectangular window has strong sidelobes. Those sidelobes produce oscillations near the ideal discontinuity. That is Gibbs behavior. The deck explicitly warns against the misconception:

“Make the filter longer and the Gibbs ripple disappears.” It does not. A longer rectangular window narrows the oscillation region, but the limiting peak overshoot does not vanish.

What is a good answer to “What does increasing N do?”

Weak answer:

“It reduces ripple.” Not quite.

Better answer:

“For rectangular truncation, increasing N makes the transition narrower.” Strong answer: “The oscillations are compressed closer to the discontinuity, but the peak Gibbs overshoot remains approximately fixed. Filter length and sidelobe level are different design controls.” That is exactly the distinction highlighted in the deck.

Slide 13: The window method

The basic window-method equation is

$$\boxed{h_w[n]h_d[n]w[n].}$$

This looks trivial in time, but the important consequence is in frequency:

$$\boxed{H_w(e^{j\omega})\frac{1}{2\pi}H_d(e^{j\omega}) * W(e^{j\omega})}$$

where the convolution is periodic in frequency. The window spectrum determines two major properties:

$$\boxed{\text{main-lobe width}}$$

and

$$\boxed{\text{sidelobe level.}}$$

A smoother taper in time reduces sidelobes but broadens the main lobe. The deck states this design intuition directly. Why does a wider window main lobe produce a wider filter transition?

The ideal lowpass has a sharp edge. When convolved with the window spectrum, that edge gets blurred. A narrow window main lobe blurs the edge less. A wide main lobe blurs it more. So:

$$\boxed{\text{narrow main lobe} \Rightarrow \text{narrow transition}}$$

but usually:

$$\boxed{\text{higher sidelobes} \Rightarrow \text{more ripple/leakage.}}$$

This is the core tradeoff.

Slides 14–16: Comparing windows

The deck compares Rectangular, Bartlett, Hann, Hamming, and Blackman. The approximate values in the lecture are:

Window	First sidelobe	Main-lobe width
Rectangular	-13 dB	$\frac{4\pi}{N}$
Bartlett	-26 dB	$\frac{8\pi}{N}$
Hann	-31 dB	$\frac{8\pi}{N}$
Hamming	-43 dB	$\frac{8\pi}{N}$
Blackman	-58 dB	$\frac{12\pi}{N}$

as approximate equal-length rules. The key message is not to memorize every number. The correct engineering principle is:

better stopband attenuation usually costs wider transition

for a fixed N. Good answer to “Which window is best?” Bad answer:

“Blackman.” That ignores the specification.

Good answer:

“There is no universally best window.”

Strong answer:

“The choice depends on whether the specification prioritizes transition width or stopband attenuation. A Blackman window gives lower sidelobes but a wider transition, while a rectangular window gives a narrower transition for the same length but much worse sidelobes.”

The deck explicitly gives that as the good answer.

Hamming versus Hann

Students often ask why these two are both needed. A useful qualitative answer:

Hann has a smooth taper to zero and good general spectral leakage behavior. Hamming adjusts the cosine weighting so the first sidelobe is significantly lower. Hamming does not go exactly to zero at the endpoints. Both have approximately similar main-lobe width for the same N, but different sidelobe behavior. The deck specifically notes Hamming's particularly low first sidelobe.

Slide 17: Practical 51-tap Hamming lowpass

The design flow students should internalize is:

$$\boxed{H_d(e^{j\omega})}$$

choose ideal specification, then

$$\boxed{h_d[n]}$$

via inverse DTFT, then choose

$$\boxed{w[n]}$$

and form

$$\boxed{h[n] = h_d[n]w[n].}$$

Then shift and verify the actual response. The deck says Hamming is chosen because we accept a somewhat wider transition in exchange for much better sidelobe rejection than rectangular truncation.

A strong student answer to

“Why Hamming?” would be:

“Because the design can tolerate some transition broadening, and the Hamming window gives significantly lower sidelobes than a rectangular window.”

Slide 18: Filter length

Increasing N generally narrows the transition for a fixed window family. But there are costs:

$$\boxed{\text{more multiplications}}$$

$$\boxed{\text{more memory}}$$

and for linear-phase FIR,

$$\boxed{\tau_g \frac{N-1}{2}}$$

increased delay. The ML analogy is also useful:

A longer Conv1d kernel gives a larger receptive field and, if trainable, more parameters.

Good example: delay cost At sampling rate

$$f_s = 48\ \text{kHz},$$

a 101-tap symmetric FIR has

$$\tau_g \frac{101-1}{2} 50$$

samples. The physical delay is

$$\frac{50}{48000} \approx 1.04\ \text{ms}.$$

For many applications that is small. But if

$$N = 4001,$$

then

$$\tau_g = 2000$$

samples, giving

$$\frac{2000}{48000} \approx 41.7\ \text{ms}.$$

Now latency may matter significantly. That is a good practical way to connect the math to implementation.

Slide 19: Choosing the right design knob

The deck gives three concept prompts.

Case A Stopband leakage is unacceptable, but a wider transition is okay. Good answer:

Use a stronger taper such as Blackman, or increase N. Why? Because a stronger window lowers sidelobes.

Case B Transition must be very narrow, but moderate ripple is acceptable. Good answer:

A rectangular window gives a narrower main lobe for the same N, or increase N.

Case C Relative waveform timing must be preserved.

Good answer:

Use a linear-phase FIR or another phase-preserving method. The phrase “another phase-preserving method” is useful because FIR is not the only possible strategy.

Slides 20–22: FIR and PyTorch Conv1d

The classical FIR equation is

$$\boxed{y[n] \sum_{k=0}^{M} h\,[k]x[n-k].}$$

That is a sliding dot product. This is the same local arithmetic used in Conv1d.

The deck's central ML bridge is exactly this: DSP designs $h[k]$, while ML can make the corresponding coefficients trainable.

Important subtlety: PyTorch uses cross-correlation ordering PyTorch Conv1d computes the deep-learning convention, which is closer to

$$y[n] \sum_{k} w\,[k]x[n+k]$$

under the simplest indexing picture. Textbook causal convolution is

$$y[n] \sum_{k} h\,[k]x[n-k].$$

Therefore, to reproduce textbook convolution with Conv1d, reverse the taps. The deck explicitly says to left-pad by M and flip the coefficient order. Good answer to “Is PyTorch doing convolution wrong?”

Bad answer:

“Yes, technically.”

Better answer:

“It uses cross-correlation ordering, which deep-learning libraries conventionally call convolution.” Strong answer:

“The local weighted-sum structure is the same. Since neural kernels are usually learned, the flip convention is not operationally important during learning; it matters mainly when reproducing a known DSP impulse response exactly.”

That is a very good senior/graduate-level explanation.

Slide 22: Fixed FIR in PyTorch

The deck uses

h = torch.tensor(h_np, dtype=torch.float32) x = torch.tensor(x_np).view(1, 1, -1)

kernel = h.flip(0).view(1, 1, -1) x_pad = F.pad(x, (len(h)-1, 0)) y = F.conv1d(x_pad, kernel) The important tensor shape is

$$(N, C, L).$$

For one mono signal:

$$\boxed{(1,1,L).}$$

The deck explains that the same coefficients can either remain fixed or become trainable depending on whether gradients are allowed to update them.

Slide 23: Tensor dimensions

For Conv1d,

$$N = \text{batch size},$$

$$C = \text{channels},$$

$$L = \text{signal length}.$$

For 32 stereo one-second clips sampled at 16 kHz,

$$\boxed{(32,2,16000).}$$

The deck treats these dimensions as carrying physical meaning, which is exactly the right way to teach it.

Slide 24: Why NumPy convolution and PyTorch agree

If the coefficient flip and padding are handled correctly, both implement the same linear system. Therefore,

$$y_{\text{NumPy}}[n] \approx y_{\text{PyTorch}}[n]$$

up to floating-point roundoff. The deck reports a mismatch on the order of

$$10^{-7}$$

for float32. The important interpretation is not the exact number. It is:

Same impulse response, same input, same signal-processing result.

Slide 25: Designed versus learned taps

This is one of the most important conceptual slides. Classical FIR design:

$$\boxed{H_d(e^{j\omega}) \rightarrow h_d[n] \rightarrow \text{window} \rightarrow h[n]}$$

The coefficients are chosen to satisfy known frequency-domain specifications. Hybrid approach: Initialize a Conv1d kernel with known DSP coefficients, then optionally fine-tune. Fully learned approach: Choose

kernel size,

number of channels,

and let backpropagation choose the coefficients. The deck correctly states that the arithmetic is the same; the source of the weights is what changes.

A very useful comparison for students

Suppose the task is denoising speech before classification.

Classical approach: Design a known lowpass or bandpass filter with specified:

$$\omega_p,$$

$$\omega_s,$$

$$A_s.$$

Learned approach: Give a Conv1d layer enough taps and let the classification loss choose the kernel.

Hybrid approach: Initialize the kernel as a speech-band FIR and let training adjust it. This lets students see three different engineering philosophies.

Slide 26: DSP versus ML vocabulary

The deck maps:

FIR filter ↔ Conv1d kernel

and

$h[k]$ ↔ kernel weight.

But it also includes a very important nuance:

windowing ≢ ML regularization.

Calling a window “kernel regularization” is only an analogy. A DSP window is a deterministic spectral-design tool.

That distinction is worth preserving.

Homework 1: Ideal lowpass derivation

A complete strong answer should show

$$h_d[n]\frac{1}{2\pi}\int_{-\omega_c}^{\omega_c} e^{j\omega n}, d\omega$$

and derive

$$\boxed{h_d[n]\frac{\sin(\omega_c n)}{\pi n}, \quad n \neq 0}$$

with

$$\boxed{h_d[0]\frac{\omega_c}{\pi}.}$$

Then explain:

infinite duration, two-sided/noncausal, must be truncated. For $N = 41$,

$$\tau_g\frac{41-1}{2}\boxed{20}.$$

The instructor solution confirms this.

Homework 2: Rectangular versus Hamming

A good answer should say: Rectangular:

narrower transition

but

higher sidelobes

with first sidelobe around

-13 dB.

Hamming:

wider transition

but much lower sidelobe around

$$-43 \text{ dB}.$$

A strong answer also says:

Increasing rectangular-window length narrows the Gibbs region but does not eliminate the peak overshoot. That is exactly the instructor solution.

Homework 3: Linear phase proof

A strong answer starts from symmetry:

$$h[n]h[N-1-n].$$

Pair symmetric terms in the DTFT. After factoring out

$$e^{-j\omega(N-1)/2},$$

the remaining amplitude is real. Thus,

$$H(e^{j\omega})A(\omega)e^{-j\ (N-1)/2}$$

and therefore

$$\boxed{\tau_g = \frac{N-1}{2}.}$$

The instructor appendix summarizes exactly this logic.

Homework 4: Convolution and kernel flip

Given

$$h = [0.2,0.5,0.3]$$

and

$$x = [1,2,0,-1],$$

the first outputs are:

$$y[0]0.2(1)\boxed{0.2}.$$

$$y[1]0.2(2) + 0.5(1)0.4 + 0.5\boxed{0.9}.$$

$$y[2]0.2(0) + 0.5(2) + 0.3(1)1 + 0.3\boxed{1.3}.$$

$$y[3]0.2(-1) + 0.5(0) + 0.3(2) - 0.2 + 0.6\boxed{0.4}.$$

So:

$$\boxed{[0.2, 0.9, 1.3, 0.4, \ldots]}$$

which matches the deck solution. The PyTorch kernel must be flipped because Conv1d uses correlation ordering.

Good exit-ticket questions for this lecture

If I wanted only three:

1. Why does a symmetric FIR have linear phase?

Good answer:

Because symmetry lets the DTFT be written as a real amplitude term multiplied by

$$e^{-j\ (N-1)/2},$$

which gives constant group delay

$$\frac{N-1}{2}.$$

1. Why does a smoother window reduce sidelobes but widen the transition?

Good answer:

Smoother time-domain tapering lowers the window spectrum's sidelobes, but its main lobe becomes wider. Since the desired response is convolved with the window spectrum, that wider main lobe broadens the filter transition.

1. What does neural Conv1d add to a classical FIR?

Good answer:

It does not change the local multiply-and-sum operation. It embeds it in a trainable model so the coefficients can be learned from a loss. That final answer is exactly the minute-paper answer in the deck.

What I would expect a senior student to know

A strong senior should be able to: derive

$$h_d[n]\frac{\sin(\omega_c n)}{\pi n},$$

explain why it must be approximated, compare window tradeoffs, compute

$$\tau_g \frac{N-1}{2},$$

and explain how Conv1d relates to FIR filtering.

What I would expect a first-year graduate student to know

In addition, I would expect them to explain:

why time-domain truncation causes frequency-domain convolution,

why Gibbs overshoot does not vanish merely by increasing rectangular-window length,

why a real symmetric FIR factors as

$$A(\omega)e^{-j\omega(N-1)/2},$$

and why the classical window method should be treated as a deterministic design constraint rather than literally equated with ML regularization.

The lecture closes with exactly these four ideas:

- symmetric FIRs provide constant group delay,
- the ideal brick-wall response produces an infinite sinc,
- windowing trades sidelobe suppression against transition width, and
- Conv1d performs the same local weighted sum with coefficients that may be fixed or learned.

Week 4, Lecture 8, LaTeX math

Explain in more detail what's going on here and what good answers and/or examples are. Please write all math expressions using LaTeX syntax.

Chat GPT: (Medium)

Please convert all math expressions to LaTeX. Don't change anything else.

Gemini Flash:

Absolutely. Lecture 8 is really about one large design story:

digital filter specification → analog prototype → bilinear transform → digital IIR → ML preprocessing cons

The lecture's deeper purpose is not just to teach a formula for converting s-domain filters into z-domain filters. It is to show that filter design choices change the data distribution seen by a machine-learning model—especially through phase, causality, and train/deployment mismatch.

Slides 3–8: Why use an IIR at all?

An IIR filter usually has a recursive form such as

$$y[n] = \sum_{k=0}^{M} b_k\, x[n-k] - \sum_{r=1}^{N} a_r\, y[n-r]$$

The feedback terms

$$y[n-r]$$

allow a relatively low-order filter to create a sharp frequency response. That is the main attraction.

For comparable magnitude specifications, an IIR filter can often use far fewer coefficients than an FIR filter. The cost is that poles now matter, numerical sensitivity matters, and the phase is generally nonlinear.

A good answer to

“Why would I choose an IIR instead of an FIR?” is:

“If I need a sharp magnitude response with low computational cost and can tolerate nonlinear phase, an IIR can be much more efficient.”

A stronger answer adds:

“But because the recursion introduces poles, I also have to worry about stability and implementation sensitivity.”

What nonlinear phase means here

Recall that group delay is

$$\boxed{\tau_g(\omega) = -\frac{d}{d\omega}\angle H(e^{j\omega})}$$

If

$$\tau_g(\omega)$$

varies strongly with ω, different frequency components are delayed by different amounts. So two filters can have almost the same

$$|H(e^{j\omega})|$$

but produce visibly different time-domain waveforms. That matters if a downstream classifier uses timing-sensitive features such as

onset shape,

peak timing,

or short-time spectral dynamics. The deck explicitly makes this ML preprocessing connection.

Slides 4–8: Analog prototypes

The classical approach is:

Instead of designing every digital IIR filter from scratch, start with a well-understood analog prototype. The deck compares four common families.

Butterworth Butterworth is maximally flat in the passband. Its magnitude-squared response is

$$\boxed{|H_c(j\Omega)|^2 = \frac{1}{1 + \left(\frac{\Omega}{\Omega_c}\right)^{2N}}}$$

where

$$N$$

is the filter order and

$$\Omega_c$$

is the cutoff frequency. At

$$\Omega = \Omega_c,$$

we get

$$|H_c(j\Omega_c)|^2 = \frac{1}{1+1} = \frac{1}{2}.$$

Therefore,

$$|H_c(j\Omega_c)| = \frac{1}{\sqrt{2}}.$$

In dB,

$$20\log_{10}\left(\frac{1}{\sqrt{2}}\right) \approx \boxed{-3.01\ \text{dB}}.$$

Importantly, that cutoff result is independent of N. The deck highlights exactly this observation.

What does increasing Butterworth order do? Far above cutoff,

$$\left(\frac{\Omega}{\Omega_c}\right)^{2N} \gg 1.$$

Then approximately,

$$|H_c(j\Omega)| \approx \left(\frac{\Omega_c}{\Omega}\right)^{N}.$$

In dB,

$$20\log_{10}|H| \approx -20N\log_{10}\left(\frac{\Omega}{\Omega_c}\right).$$

So every additional pole contributes about

$$\boxed{\text{20 dB/decade}}$$

of asymptotic attenuation.

The deck also warns that higher order increases implementation complexity and numerical sensitivity.

Good question: Why not always use very high order?

Weak answer:

“Because it costs more.”

Better answer:

“Higher order gives a sharper transition, but it increases implementation complexity and can make direct-form coefficient realizations numerically sensitive.”

Strong answer:

“For moderate or high orders, I should generally implement the digital IIR as cascaded second-order sections rather than one large direct-form polynomial.”

That is exactly the implementation habit recommended in the deck.

Butterworth, Chebyshev, elliptic: what are we trading?

The deck's table is really about where we allow approximation error.

Butterworth

Passband:

monotonic

Stopband:

monotonic

Advantage: smooth response.

Chebyshev I

Passband:

equiripple

Stopband:

monotonic

The allowed passband ripple buys a sharper transition.

Chebyshev II

Passband:

monotonic

Stopband:

equiripple

Now the ripple is spent in the stopband.

Elliptic

Passband:

equiripple

Stopband:

equiripple

This often gives the sharpest transition for a specified order.

Good answers to the prototype concept check The deck gives three scenarios.

Smooth calibration response

If passband amplitude must be smooth and some transition width is acceptable:

Butterworth

is a good first choice.

Tight transition, some passband ripple allowed

Chebyshev I

is reasonable.

Minimum order is important and ripple is acceptable in both bands

Elliptic

is often the most aggressive classical choice.

The important answer is not just the filter name. Students should state which constraint they are spending.

Slides 9–12: What must the analog-to-digital map preserve?

The deck asks what a useful mapping from the analog s-plane to the digital z-plane should preserve. We want at least three things.

1. Stability Analog stability requires

$$\mathrm{Re}\{s\} < 0.$$

Digital stability requires poles to satisfy

$$|z| < 1.$$

So the left-half s-plane should map inside the unit circle.

1. Frequency axis The analog frequency axis is

$$s = j\Omega.$$

We want it to map to the digital unit circle

$$z = e^{j\omega}.$$

1. One-to-one frequency mapping We do not want infinitely many analog frequencies folding onto one digital frequency. The bilinear transform accomplishes this—but frequency spacing becomes nonlinear.

Slide 10: Bilinear transform

The substitution is

$$\boxed{s = \frac{2}{T}\frac{1 - z^{-1}}{1 + z^{-1}}}$$

where

$$T$$

is the sampling period associated with the mapping. The inverse form is especially useful for understanding pole mapping:

$$\boxed{z = \frac{1 + \frac{sT}{2}}{1 - \frac{sT}{2}}}$$

The deck correctly distinguishes this from the impulse-invariance relationship

$$z = e^{sT}.$$

The bilinear transform avoids analog-frequency aliasing, but introduces warping instead.

Slides 11–12: Why stability is preserved

Let

$$s = \sigma + j\Omega.$$

Then

$$z = \frac{1 + \frac{sT}{2}}{1 - \frac{sT}{2}}.$$

The squared magnitude is

$$|z|^2 = \frac{\left(1 + \frac{\sigma T}{2}\right)^2 + \left(\frac{\Omega T}{2}\right)^2}{\left(1 - \frac{\sigma T}{2}\right)^2 + \left(\frac{\Omega T}{2}\right)^2}.$$

If the analog pole is stable,

$$\sigma < 0.$$

Then

$$\left|1 + \frac{\sigma T}{2}\right| < \left|1 - \frac{\sigma T}{2}\right|$$

in the relevant magnitude comparison, giving

$$\boxed{|z| < 1.}$$

So the stable analog left-half plane maps inside the digital unit circle. The deck states this as the key guarantee.

Boundary case If

$$\sigma = 0,$$

then

$$s = j\Omega.$$

The numerator and denominator have equal magnitude, so

$$\boxed{|z| = 1}.$$

Therefore the analog frequency axis maps onto the digital unit circle. That gives the desired boundary mapping.

Homework pole-mapping example

The deck later uses

$$s = -2 + j3, \qquad T = 0.1.$$

Then

$$\frac{sT}{2} = \frac{(-2 + j3)(0.1)}{2} = -0.1 + j0.15.$$

Therefore,

$$z = \frac{1 - 0.1 + j0.15}{1 + 0.1 - j0.15}.$$

Numerically,

$$\boxed{z \approx 0.785 + j0.243}$$

and

$$|z| \approx \boxed{0.822 < 1}.$$

So a stable analog pole mapped to a stable digital pole, exactly as expected.

Slides 13–15: Frequency mapping

Set

$$s = j\Omega$$

and

$$z = e^{j\omega}.$$

Substitute into the bilinear transform:

$$j\Omega = \frac{2}{T}\frac{1 - e^{-j\omega}}{1 + e^{-j}}.$$

Multiply numerator and denominator by

$$e^{j\omega/2}.$$

Then

$$1 - e^{-j\omega} = e^{-j\ /2}\left(e^{j\omega/2} - e^{-j\omega/2}\right),$$

and

$$1 + e^{-j\omega} = e^{-j\ /2}\left(e^{j\omega/2} + e^{-j\omega/2}\right).$$

Using

$$e^{j\theta} - e^{-j\theta} = 2j\sin\theta$$

and

$$e^{j\theta} + e^{-j\ } = 2\cos\theta,$$

we get

$$j\Omega = \frac{2}{T}j\tan\left(\frac{\omega}{2}\right).$$

Therefore,

$$\boxed{\Omega = \frac{2}{T}\tan\left(\frac{\omega}{2}\right).}$$

This is the central warping equation of the lecture.

What the mapping means

At

$$\omega = 0,$$

$$\Omega = 0.$$

For small ω,

$$\tan\left(\frac{\omega}{2}\right) \approx \frac{\omega}{2},$$

so

$$\Omega \approx \frac{\omega}{T}.$$

Thus low frequencies map approximately linearly. But as

$$\omega \to \pi,$$

$$\tan\left(\frac{\omega}{2}\right) \to \infty.$$

So

$$\boxed{\omega \to \pi \quad \Rightarrow \quad \Omega \to \infty.}$$

That is why the whole analog frequency axis fits into

$$-\pi < \omega < \pi.$$

The price is severe frequency compression near Nyquist.

Good answer to “What is frequency warping?”

Weak answer:

“The frequencies change.”

Better answer:

“The bilinear transform maps analog frequency to digital frequency nonlinearly.”

Strong answer:

“Equal spacing in analog frequency does not remain equally spaced digitally. The mapping is nearly linear near DC but becomes strongly compressed as digital frequency approaches π.”

Slides 16–18: Prewarping

Suppose the digital specification requires a cutoff at

$$\omega_c.$$

We cannot simply design the analog filter at

$$\Omega_c = \frac{\omega_c}{T}.$$

Instead, use

$$\boxed{\Omega_c = \frac{2}{T}\tan\left(\frac{\omega_c}{2}\right).}$$

That is prewarping.

The deck gives the design sequence:

$$\boxed{\text{digital spec} \rightarrow \text{prewarp} \rightarrow \text{analog prototype} \rightarrow \text{bilinear transform.}}$$

Worked example: $\omega_c = 0.4\pi$ Let

$$T = 1$$

and

$$\omega_c = 0.4\pi.$$

Then

$$\Omega_c = 2\tan\left(\frac{0.4\pi}{2}\right)$$

so

$$\Omega_c = 2\tan(0.2\pi).$$

Numerically,

$$\tan(0.2\pi) \approx 0.7265.$$

Therefore,

$$\boxed{\Omega_c \approx 1.4531.}$$

Notice that

$$0.4\pi \approx 1.2566.$$

So using the naive value

$$1.2566$$

for the analog cutoff would miss the desired digital cutoff. The deck emphasizes this exact comparison.

Good answer to “Why do we prewarp?”

“Because the bilinear transform preserves ordering but not linear frequency spacing. Prewarping chooses the analog critical frequency whose nonlinear mapping lands exactly at the desired digital critical frequency.”

That is the key idea.

Multiple critical frequencies

For a bandpass or bandstop specification, there may be several important frequencies:

$$\omega_1, \omega_2, \ldots.$$

Each should be mapped independently:

$$\boxed{\Omega_i = \frac{2}{T}\tan\left(\frac{\omega_i}{2}\right).}$$

That is an important extension beyond the single-cutoff example.

Slides 18–20: Full design workflow

The deck summarizes the practical workflow as: Start with digital specifications:

$$\omega_p, \qquad \omega_s, \qquad A_p, \qquad A_s.$$

Prewarp:

$$\omega_p \to \Omega_p$$

and

$$\omega_s \to \Omega_s.$$

Design

$$H_c(s).$$

Apply the bilinear transform to get

$$H(z).$$

Finally:

$$\boxed{\text{verify the actual digital response.}}$$

That final step matters. The bilinear transform guarantees the mapping mathematics, but numerical implementation and design choices still need checking.

Why show the explicit SciPy path? The deck uses something like

```
wc = 0.4*np.pi
T = 1.0
Omega_c = (2/T)*np.tan(wc/2)

b_a, a_a = signal.butter(4, Omega_c, analog=True)
b_z, a_z = signal.bilinear(b_a, a_a, fs=1/T)
w, H = signal.freqz(b_z, a_z)
```

The benefit is pedagogical. Students can see:

$$\boxed{\text{prewarp}}$$

and

$$\boxed{\text{analog prototype}}$$

as explicit steps rather than hiding everything inside one convenience function.

Verification result The target is

$$\omega_c = 0.4\pi.$$

The script finds approximately

$$\boxed{\omega_{-3\,\text{dB}} \approx 0.3999\pi.}$$

That is excellent evidence that the prewarp step did what it was supposed to do.

Slides 21–22: IIR phase

The same efficient recursive structure that gives sharp magnitude response usually creates nonlinear phase. So

$$\angle H(e^{j\omega})$$

is not linear in ω. Therefore,

$$\tau_g(\omega) = -\frac{d}{d\omega}\angle H(e^{j\omega})$$

is not constant.

A good class question is:

“If the magnitude response is exactly what I want, why should I care about phase?”

Good answer:

“Because a signal is reconstructed from both magnitude and phase relationships among frequency components. Frequency-dependent delay can change waveform shape even if the spectral magnitude looks good.”

ML example Suppose you're classifying impacts using onset features. If low frequencies are delayed by

$$4 \text{ samples}$$

but high frequencies by

$$18 \text{ samples},$$

the shape of the attack changes. If the classifier uses only long-term average spectral energy, perhaps that does not matter. If it uses transient timing, it might matter a lot. This motivates the deck's practical question:

Does the downstream model care about absolute timing, relative timing, or only long-term spectral statistics?

Slides 23–25: Forward-backward filtering

This is one of the most important ML-preprocessing sections. Suppose one causal filtering pass has frequency response

$$H(e^{j\omega}).$$

Filtering backward produces the conjugate-frequency counterpart for real-coefficient filters, so the combined response is

$$\boxed{H_{\text{fb}}(e^{j\omega}) = H(e^{j\omega})H(e^{-j\omega}).}$$

For real-coefficient filters,

$$H(e^{-j\omega}) = H^*(e^{j\omega}),$$

so

$$H_{\text{fb}}(e^{j\omega}) = H(e^{j\omega})H^*(e^{j\omega}).$$

Therefore,

$$\boxed{H_{\text{fb}}(e^{j\omega}) = |H(e^{j\omega})|^2.}$$

This is the key result behind `filtfilt`.

Why does phase cancel? Write

$$H(e^{j\omega}) = |H(e^{j\omega})|e^{j\phi(\omega)}.$$

Then

$$H(e^{-j\ \ }) = |H(e^{j\omega})|e^{-j\ \ (\omega)}.$$

Multiply:

$$H_{\text{fb}} = |H|^2 e^{j\phi}e^{-j\phi}.$$

Therefore,

$$\boxed{\angle H_{\text{fb}} = 0.}$$

So the net phase distortion cancels.

But the magnitude does not stay the same A very common misconception is:

“`filtfilt` gives me the same filter but with zero phase.”

Not exactly. One pass:

$$|H|.$$

Forward-backward:

$$\boxed{|H|^2}.$$

If one pass gives

$$-20 \text{ dB}$$

at some frequency, two passes give approximately

$$\boxed{-40 \text{ dB}}.$$

Because

$$20\log_{10}|H|^2 = 40\log_{10}|H|.$$

The deck makes this warning explicit.

Good answer to “What does `filtfilt` buy?”

Weak answer:

“Zero phase.”

Better answer:

“It cancels phase distortion.”

Strong answer:

“Forward-backward filtering cancels the net phase for a real-coefficient filter, but it also squares the magnitude response and is noncausal because the backward pass uses future samples.” That is the full answer.

Why is `filtfilt` noncausal? A real-time causal system at time n may only depend on

$$x[k], \qquad k \le n.$$

But the backward pass processes the entire record in reverse. Therefore the final output at a point can depend on samples that were originally in the future. So:

$$\boxed{\text{forward-backward filtering is fundamentally offline.}}$$

The deck makes this explicit.

Slides 26–27: Why this matters for ML

This is arguably the most important part for the combined DSP/ML course. Suppose you create the training dataset using

`signal.filtfilt(...)`

Then the features are based on a future-aware zero-phase preprocessing pipeline. Later you deploy a classifier in a streaming system. Now only a causal filter is available. Then:

$$\boxed{p_{\text{train}}(\mathbf{x}) \neq p_{\text{deploy}}(\mathbf{x})}$$

because the preprocessing itself has changed. This is train/serve skew. The deck warns about exactly this issue.

Good answer to the minute-paper question

The wrap-up asks:

Why can zero-phase offline preprocessing give misleading confidence about streaming deployment? A strong answer:

“Because forward-backward filtering uses future samples and produces a different effective magnitude response than a causal one-pass filter. A classifier trained on those offline features may therefore see a different input distribution when deployed in real time.”

That is the answer I would want.

Boundary effects `filtfilt` also has finite-record edge issues. At the beginning of a record, the filter does not have an infinite history. At the end, the backward pass has an analogous problem. So practical implementations use padding or special initial conditions. Therefore beginning/end samples can behave differently from the interior. The deck explicitly lists boundary transients as a caveat.

Slide 27: Where a fixed IIR fits in an ML pipeline

The deck proposes:

$$\boxed{\text{raw signal} \rightarrow \text{fixed IIR} \rightarrow \text{DSP features} \rightarrow \text{classifier}}$$

where the classifier could be:

$$\text{SVM}, \quad \text{random forest}, \quad \text{MLP}.$$

A reasonable use case:

There is a known narrow interference band that is irrelevant to the classification task. Then removing it before feature extraction may improve robustness. But the important question is:

Is it really nuisance energy?

If the supposedly unwanted band actually carries class information, filtering can hurt performance. So preprocessing is an inductive assumption.

Good answer to “Should I always clean the signal before ML?”

No.

A strong answer is:

“Only if I have reason to believe the removed component is nuisance variation rather than predictive signal. The assumption should be validated on held-out data.”

That is a much better ML engineering answer than “yes, cleaner is better.”

Slide 28: FIR, IIR, `filtfilt`, or learned front end?

The deck's comparison is useful.

Linear-phase FIR

Strength:

controlled phase

and finite support.

Cost:

more taps

and delay. Best when waveform alignment matters.

IIR

Strength:

sharp response with few coefficients.

Cost:

nonlinear phase and feedback. Best when efficient causal preprocessing matters.

`filtfilt` IIR

Strength:

zero-phase offline filtering.

Cost:

noncausal,

$|H|^2$,

and edge artifacts. Best for offline dataset preparation when deployment assumptions match.

Learned Conv1d

Strength:

task-adaptive front end.

Cost:

needs data and validation. Best for end-to-end learning.

Homework 5: Butterworth

For a fourth-order Butterworth,

$$N = 4.$$

At

$$\Omega = \Omega_c,$$

$$|H|^2 = \frac{1}{1 + 1^8} = \frac{1}{2}.$$

Thus,

$$\boxed{|H| = \frac{1}{\sqrt{2}}}$$

and

$$\boxed{20\log_{10}|H| \approx -3.01 \text{ dB}.}$$

At

$$\Omega = 2\Omega_c,$$

$$|H|^2 = \frac{1}{1 + 2^8} = \frac{1}{257}.$$

So

$$|H| = \frac{1}{\sqrt{257}} \approx 0.0624.$$

Therefore,

$$20\log_{10}(0.0624) \approx \boxed{-24.1\ \text{dB}}.$$

This matches the instructor solution.

Homework 6: Stability mapping

Given

$$s = -2 + j3, \qquad T = 0.1,$$

use

$$z = \frac{1 + sT/2}{1 - sT/2}.$$

The result is approximately

$$\boxed{z \approx 0.785 + j0.243}$$

with

$$\boxed{|z| \approx 0.822.}$$

Since

$$|z| < 1,$$

the digital pole is stable.

Homework 7: Prewarp

Given

$$\omega_c = 0.4\pi$$

and

$$T = 1,$$

$$\Omega_c = 2\tan(0.2\pi).$$

Therefore,

$$\boxed{\Omega_c \approx 1.4531.}$$

This is the analog cutoff that should be used for the prototype.

Homework 8: `filtfilt`

A full-credit answer should mention four things:

Phase cancellation:

$$\angle H_{\text{fb}} = 0.$$

Magnitude:

$$\boxed{|H_{\text{fb}}| = |H|^2.}$$

Noncausality: future samples are implicitly used.

ML risk: offline training features may not match causal deployment features. The instructor solution states these same points.

Quiz Q5: Butterworth cutoff

At

$$\Omega = \Omega_c,$$

$$|H| = \frac{1}{\sqrt{2}}$$

so

$$\boxed{-3.01 \text{ dB}}.$$

Quiz Q6: Warping equation

$$\boxed{\Omega = \frac{2}{T}\tan\left(\frac{\omega}{2}\right)}$$

This should become almost automatic by the end of the lecture.

Quiz Q7: Should $\Omega_c = 0.6\pi/T$?

No. For

$$\omega_c = 0.6\pi,$$

prewarp:

$$\boxed{\Omega_c = \frac{2}{T}\tan(0.3\pi).}$$

The reason is that the bilinear transform's frequency mapping is nonlinear.

Quiz Q8: Main benefit and limitation of `filtfilt`

Benefit:

$$\boxed{\text{zero net phase distortion}}$$

Limitation:

$$\boxed{\text{noncausal / future-aware}}$$

A stronger answer adds:

The magnitude is also squared, and edge handling can matter.

One subtle point worth emphasizing: **bilinear transform versus impulse invariance**

Students often mix these up. Impulse invariance approximately samples the analog impulse response:

$$h_d[n] \propto h_a(nT).$$

This gives pole mapping associated with

$$z = e^{sT}.$$

The disadvantage is frequency aliasing of the analog spectrum. The bilinear transform instead uses

$$s = \frac{2}{T}\frac{1 - z^{-1}}{1 + z^{-1}},$$

which creates a one-to-one analog-to-digital frequency map. So:

$$\boxed{\text{impulse invariance} \rightarrow \text{possible frequency aliasing}}$$

whereas

bilinear transform → frequency warping, no analog-frequency aliasing.

That contrast helps students remember why prewarping exists.

Another useful question: **Does the bilinear transform preserve filter type?**

For the usual lowpass prototype transformation, yes in the qualitative sense: DC maps to DC and infinite analog frequency maps toward Nyquist. But exact frequency spacing is not preserved. So a better answer is:

"It preserves the ordering and one-to-one nature of the frequency axis, but not linear spacing."

Good exit-ticket questions

If you want only three for Lecture 8, I would use these.

1. Why does the bilinear transform preserve stability?

Good answer:

Analog poles with

$$\mathrm{Re}\{s\} < 0$$

map to digital poles satisfying

$$|z| < 1.$$

For the inverse map

$$z = \frac{1 + sT/2}{1 - sT/2},$$

the negative real part makes the numerator magnitude smaller than the denominator magnitude.

1. Why is prewarping necessary?

Good answer:

Because

$$\Omega = \frac{2}{T}\tan\left(\frac{\omega}{2}\right)$$

is nonlinear, so an unwarped analog cutoff does not map to the intended digital cutoff.

1. Why can `filtfilt` create a problem for ML deployment?

Good answer:

It is noncausal and future-aware, so a model trained on forward-backward-filtered data may see a different feature distribution when deployed with real-time causal preprocessing.

A strong answer also mentions

$$|H|^2$$

and boundary effects.

What I would expect a senior student to understand

A strong senior should be able to: compare the classical analog prototype families, use

$$|H_c(j\Omega)|^2 = \frac{1}{1 + \left(\frac{\Omega}{\Omega_c}\right)^{2N}},$$

explain why the bilinear transform preserves stability, derive or use

$$\Omega = \frac{2}{T}\tan\left(\frac{\omega}{2}\right),$$

perform a simple prewarp calculation, and explain the difference between causal IIR filtering and `filtfilt`.

What I would expect a first-year graduate student to understand

In addition, I would want them to explain:

- why the analog $j\Omega$ axis maps onto the digital unit circle,
- why the entire infinite analog frequency axis fits into

$$-\pi < \omega < \pi,$$

- why the bilinear transform eliminates analog-frequency aliasing at the cost of warping,
- why second-order sections are preferable for higher-order implementations, and
- why preprocessing assumptions are part of the ML model even when the preprocessing itself is “classical DSP.”

The one-minute synthesis

A strong student summary would be:

“IIR filters use feedback to achieve sharp magnitude selectivity efficiently, but their phase is generally nonlinear. Classical analog prototypes such as Butterworth, Chebyshev, and

elliptic filters provide systematic magnitude-response tradeoffs. The bilinear transform maps a stable analog prototype into a stable digital filter, but the analog and digital frequency axes are related nonlinearly, so critical frequencies must be prewarped. In ML preprocessing, causal IIR filters can reshape timing, while forward-backward filtering cancels phase at the cost of noncausality and a squared magnitude response, so preprocessing must be consistent between training and deployment."

That captures the lecture very well and matches the wrap-up themes in the deck.

Yes. For these two lectures, I would use exit tickets that test the **main design insight** from each class rather than repeat the weekly quiz.

Lecture 7 centers on linear phase, the ideal-to-practical FIR transition, the window tradeoff, and the DSP-to-`Conv1d` bridge. Lecture 8 centers on analog-prototype tradeoffs, stability-preserving bilinear mapping, prewarping, and the implications of causal versus zero-phase preprocessing for ML.

Lecture 7 — FIR Filter Design, Windowing, & 1D Convolutions

1. Why does a symmetric FIR preserve waveform alignment better than a nonlinear-phase filter?

Question

A symmetric FIR filter has linear phase. What does that imply physically for a transient signal?

Good answer

For a symmetric FIR,

$$\tau_g = \frac{N-1}{2}$$

is constant, so frequency components in the passband experience the same delay. Their relative timing is preserved, which reduces waveform smearing.

Strong answer

> Linear phase does not mean zero delay; it means approximately constant delay versus frequency. That can preserve onset shape and peak timing before ML features are extracted.

---

2. Why can't we implement the ideal lowpass filter directly?

Question

The ideal lowpass filter has

$$h_d[n] = \frac{\sin(\omega_c n)}{\pi n}.$$

Why is this not directly implementable in a real-time system?

Good answer

Because the impulse response is both infinite in duration and two-sided. A finite processor cannot use infinitely many taps, and negative-time coefficients imply noncausality.

---

3. What actually causes Gibbs ripple?

Question

Why does rectangular truncation of the ideal sinc create ripple near the cutoff?

Good answer

Rectangular truncation is multiplication in time, which corresponds to convolution in frequency. The rectangular window has strong sidelobes, so convolving its spectrum with the ideal brick-wall response produces oscillatory ripple near the discontinuity.

Strong answer

> Increasing filter length narrows the region of oscillation but does not make the peak Gibbs overshoot vanish.

---

4. Which window would you choose?

Question

Suppose stopband leakage is very costly, but a wider transition band is acceptable. Would you prefer Rectangular, Hamming, or Blackman?

Answer

Blackman

is a reasonable choice because it has substantially lower sidelobes, at the cost of a wider main lobe and therefore a wider transition.

Good explanation

> There is no universally best window. The specification determines which tradeoff matters most.

5. What does increasing FIR length buy?

Question

If you increase N while keeping the same window family, what usually improves, and what gets worse?

Good answer

Increasing N generally narrows the transition band, but increases:

computation,

memory,

and for a linear-phase FIR,

$$\tau_g = \frac{N-1}{2}.$$

So delay increases too.

---

6. Why does PyTorch require a kernel flip for textbook convolution?

Question

Why do fixed FIR taps need to be reversed when using `torch.nn.functional.conv1d` to reproduce textbook convolution?

Good answer

Textbook convolution uses

$$y[n] = \sum_k h[k]x[n-k],$$

while PyTorch `Conv1d` uses a cross-correlation ordering. Therefore the FIR coefficients must be reversed to reproduce textbook convolution exactly.

---

7. What is the real DSP-to-ML connection?

Question

What does a neural 1-D convolution add to the FIR operation itself?

Best answer

> It does not change the local multiply-and-sum operation. It places that operation inside a trainable model so the coefficients can be learned from a loss rather than fixed by classical filter design.

That is essentially the lecture's own minute-paper answer.

Lecture 8 — IIR Design, Bilinear Transformation, & Data Preprocessing

1. Why use an IIR filter instead of an FIR?

Question

What is the main advantage and main cost of an IIR filter?

Good answer

An IIR can achieve sharp magnitude selectivity with relatively few coefficients, but the recursive denominator introduces poles and usually nonlinear phase.

---

2. Which analog prototype fits this requirement?

Question

You need a very smooth passband and can tolerate a wider transition. Which prototype is a natural first choice?

Answer

Butterworth

because it is monotonic and maximally flat in the passband.

Extension

If some passband ripple is acceptable to sharpen the transition:

Chebyshev I

If ripple is acceptable in both bands and minimum order matters:

Elliptic.

---

3. Why does the bilinear transform preserve stability?

Question

Why does a stable analog pole map to a stable digital pole under the bilinear transform?

Good answer

The inverse bilinear map is

$$z = \frac{1 + \frac{sT}{2}}{1 - \frac{sT}{2}}.$$

If

$$\mathrm{Re} s < 0,$$

then the mapped point satisfies

$$|z| < 1.$$

Therefore the analog left-half plane maps inside the digital unit circle.

---

4. Why is prewarping necessary?

Question

Why can't you just set the analog cutoff numerically equal to the desired digital cutoff?

Good answer

Because the bilinear frequency relation is nonlinear:

$$\boxed{\Omega = \frac{2}{T}\tan\left(\frac{\omega}{2}\right)}$$

so the desired digital edge must be mapped to its corresponding analog frequency before designing the prototype.

---

5. Quick prewarp check

Question

For

$$\omega_c = 0.4\pi, \qquad T = 1,$$

should the analog prototype use

$$\Omega_c = 0.4\pi?$$

Answer

No.

Use

$$\Omega_c = 2\tan(0.2\pi) \approx \boxed{1.4531}.$$

The lecture's worked design shows that this maps back to approximately the intended digital cutoff after the bilinear transform.

---

6. What does `filtfilt` actually do?

Question

What are the two main frequency-domain consequences of forward-backward filtering?

Good answer

For real-coefficient filters,

$$\boxed{H_{\text{fb}}(e^{j\omega}) = |H(e^{j\omega})|^2}$$

so:

- phase cancels,
- magnitude is squared.

Thus the attenuation in dB is approximately doubled.

---

7. Why can zero-phase preprocessing be dangerous for ML deployment?

Question

Why might a classifier trained on `filtfilt`-preprocessed signals perform differently when deployed in real time?

Good answer

Because `filtfilt` is noncausal and uses future samples. A streaming deployment may only have causal preprocessing, so the model may see a different feature distribution at deployment than during training.

Strong answer

> This is a train/serve mismatch caused by preprocessing, not by the classifier itself.

---

8. Should you always filter “noise” before ML?

Question

A narrowband component looks like interference. Should you automatically remove it before feature extraction?

Good answer

No. First ask whether that component could contain class information. Fixed filtering is an inductive assumption and should be validated on held-out data.

My preferred three-question exit ticket for each lecture

For **Lecture 7**, I would use:

1. Why does symmetric FIR structure give constant group delay?
2. Why does a smoother window lower sidelobes but widen the transition?
3. What does `Conv1d` add to the FIR operation itself?

For **Lecture 8**, I would use:

1. Why does the bilinear transform preserve stability?
2. Why must the cutoff be prewarped?
3. Why can `filtfilt` create a train/deployment mismatch in an ML pipeline?

Those six questions are short enough for roughly **4–6 minutes total per lecture**, but they reveal whether students understood the central engineering ideas rather than merely memorized formulas.
